

Excel Single-Table Multi-Period Accounting Workbook: User Guide

Leonard Brown

June 18, 2026

Contents

1	Introduction	1
1.1	Entity Description	2
2	Workbook tabs	3
3	Chart of accounts	4
4	Transaction labels (Time-Labels)	4
5	Single-Entry Accounting System (Transaction Matrix)	6
6	Double-entry sheet	6
6.1	Booking Transactions - Single Entry and Double-Entry	6
6.2	Stock Positions and Reports	8
6.3	Intra-Period versus End-of-Period Balance Sheet	9
7	Traditional Journal and Ledger	10
8	Modern Spreadsheet Functions	10

1 Introduction

This user guide illustrates the closed-flow matrix framework using a standard five-account-type accounting example. The example involves a sole proprietor who begins a small side business under a fixed contract. The business starts with no assets, liabilities, or capital, and an opening bank balance of zero.

Separate worksheets demonstrate both single-entry and double-entry implementations of the accounting system. As expected, the resulting account positions, income statements, and balance sheets are identical under both representations. The principal difference is that the double-entry implementation records gross inflows (credits) and gross outflows (debits), whereas the single-entry implementation records the corresponding net flows. Consequently, both systems contain the same underlying accounting information, but represent that information in different ways.

The purpose of this guide is to explain the structure and operation of the workbook, including the transaction matrices, reporting functions, account positions, financial statements, and supporting worksheets used throughout the example.

1.1 Entity Description

Entity Description Sole proprietor starts a side business under a fixed contract.

- Weekly revenue (Month 1): 1,000
- Weekly operating expenses (Month 1): 800
- Weekly owner draw (paid to himself): 150 (constant over time)

Starting Position (Day 1)

- No initial assets, liabilities, or capital.
- Initial Cash in bank: 0.

Bookkeeping Assumptions

- Proprietor posts one transaction per week.
- For every post, Joe is able to produce an Income Statement and an intra-period Balance Sheet.
- Proprietor closes the books at each month-end and produces an end-of-period Balance Sheet.
- Within each month, weekly revenue and weekly expenses are constant.
- Revenues and expenses increase by 10% at the start of each new month.
- The 150 per week owner draw does not increase.
- Monthly, the `OWNER_DRAW` account is consolidated into the `JOES_CAP` account.
- At the end of the fourth month, Proprietor takes a one-time draw of 1,000.

- We will show account positions, Income Statements, and Balance Sheets for four months.

2 Workbook tabs

This spread sheet has seven Tabs

1. **READ-ME-FIRST**

Provides a brief introduction to the workbook together with instructions for getting started. It explains the purpose of the spreadsheet and outlines how the various worksheets interact to generate accounting reports.

2. **ACC SYSTEM SPEC.**

Describes the accounting system implemented in the workbook. This sheet specifies the assumptions, transaction structure, reporting periods, and other parameters used in the illustrative accounting example.

3. **CHART-OF-ACCOUNTS**

Defines the chart of accounts used by the system. Each account is mapped to its account type and to the corresponding terminal path within the recursive partition hierarchy represented by the transaction matrix.

4. **SINGLE-ENTRY-SYSTEM**

Implements the single-entry version of the accounting system. Using the underlying transaction matrix, this sheet automatically generates account positions, income statements, and balance sheets for any selected reporting date. The descriptive information for each transaction is included in each row as an excel note.

5. **DOUBLE-ENTRY-SYSTEM**

Implements the double-entry version of the accounting system. This sheet provides the same reporting functionality as the single-entry system while representing transactions using a positive–negative decomposition that preserves gross inflows and outflows separately. The descriptive information for each transaction is included in each row as an excel note.

6. JOURNAL

A traditional journal representation of the journal entries generated by this entity over four months. Transactions are displayed in chronological order using conventional debit and credit entries together with descriptive information.

7. LEDGER

A traditional ledger representation from the journal data. Transactions are grouped by account and accumulated through time to produce running account balances.

3 Chart of accounts

Table 1: Chart of Accounts and Terminal Paths

Account Name	Type	Terminal Path (Transaction Matrix Column)
REVENUE	Income	Closed-System => INFLOWS => Income => REVENUE
TRANSPORT	Expense	Closed-System => OUTFLOWS => Expense => TRANSPORT
OWN_DRAW	Capital	Closed-System => INFLOWS => Capital => OWN_DRAW
BANK	Asset	Closed-System => OUTFLOWS => Asset => BANK
PROP_CAP	Capital	Closed-System => INFLOWS => Capital => PROP_CAP

Note: Each account is associated with a terminal path originating at the root node **Closed-System**. The terminal path uniquely identifies the account's location within the recursive partition hierarchy and encodes the complete ancestry of the account from the root node to the terminal node. Parent accounts and higher-level reporting categories are obtained through aggregation of terminal paths sharing a common prefix.

4 Transaction labels (Time-Labels)

The spreadsheet records transactions over a four-month period. To identify the timing of each transaction, a compact notation is used. For example, **m1.w2** denotes the transaction occurring in Month1, Week2, while **m3.close** denotes the closing entry for Month 3.

Although the notation is largely self-explanatory, Table 2 provides a complete description of all transaction labels used in the example. These labels serve as the row identifiers of the transaction matrix and establish the chronological ordering of events within the accounting system.

Table 2: Transaction labels used in the multi-period accounting system.

Label	Meaning
<code>inception</code>	Initial state of the accounting system prior to any transactions.
<code>m1.w1</code>	Month 1, Week 1 transaction.
<code>m1.w2</code>	Month 1, Week 2 transaction.
<code>m1.w3</code>	Month 1, Week 3 transaction.
<code>m1.w4</code>	Month 1, Week 4 transaction.
<code>m1.adj1</code>	Month 1 adjusting entry.
<code>m1.close</code>	Month 1 closing entry.
<code>m2.w1</code>	Month 2, Week 1 transaction.
<code>m2.w2</code>	Month 2, Week 2 transaction.
<code>m2.w3</code>	Month 2, Week 3 transaction.
<code>m2.w4</code>	Month 2, Week 4 transaction.
<code>m2.adj1</code>	Month 2 adjusting entry.
<code>m2.close</code>	Month 2 closing entry.
<code>m3.w1</code>	Month 3, Week 1 transaction.
<code>m3.w2</code>	Month 3, Week 2 transaction.
<code>m3.w3</code>	Month 3, Week 3 transaction.
<code>m3.w4</code>	Month 3, Week 4 transaction.
<code>m3.adj1</code>	Month 3 adjusting entry.
<code>m3.close</code>	Month 3 closing entry.
<code>m4.w1</code>	Month 4, Week 1 transaction.
<code>m4.w2</code>	Month 4, Week 2 transaction.
<code>m4.w3</code>	Month 4, Week 3 transaction.
<code>m4.w4</code>	Month 4, Week 4 transaction.
<code>m4.w4.1</code>	Additional one-time owner draw transaction posted after Week 4.
<code>m4.adj1</code>	Month 4 adjusting entry.
<code>m4.close</code>	Month 4 closing entry.

5 Single-Entry Accounting System (Transaction Matrix)

Single-Entry Accounting System (Transaction Matrix)

SELECT REPORT	ACCOUNTS = _i TYPE = _i	BANK ΔA	TRANSPORT ΔE	OWN_DRAW ΔC	REVENUE ΔI	PROP_CAP ΔC	
							CHECK
☐	inception	0	0	0	0	0	0
☐	m1.w1	50	800	-150	1,000		0
☐	m1.w2	50	800	-150	1,000		0
☐	m1.w3	50	800	-150	1,000		0
☐	m1.w4	50	800	-150	1,000		0
☐	m1.adj1			600		-600	0
☐	m1.close		-3,200		-4,000	800	0

Table 3: Month 1 transaction matrix for Joe's Side Business. The checkboxes in the *Select Report* column identify rows that may be selected for the generation of income statements and balance sheets. The *Check* column verifies that each transaction satisfies the closure constraint.

6 Double-entry sheet

Double-Entry Accounting System (Transaction Matrix)

SELECT REPORT	TIME	BANK $\Delta A^+ \Delta A^-$		TRANSPORT $\Delta E^+ \Delta E^-$		OWN_DRAW $\Delta C^+ \Delta C^-$		REVENUE $\Delta I^+ \Delta I^-$		PROP_CAP $\Delta C^+ \Delta C^-$				
		DR	CR	DR	CR	CR	DR	CR	DR	CR	DR	DR	CR	DIFF
☐	inception	0	0	0	0	0	0	0	0	0		0	0	0
☐	m1.w1	50		800			150	1,000				1,000	1,000	0
☐	m1.w2	50		800			150	1,000				1,000	1,000	0
☐	m1.w3	50		800			150	1,000				1,000	1,000	0
☐	m1.w4	50		800			150	1,000				1,000	1,000	0
☐	m1.adj1					600				600		600	600	0
☐	m1.close				3200			4000		800		4,000	4,000	0

Table 4: Month 1 double-entry transaction matrix for Joe's Side Business. The checkboxes in the *Select Report* column identify rows that may be selected for the generation of income statements and balance sheets. The final three columns verify that total debits equal total credits for each transaction.

6.1 Booking Transactions - Single Entry and Double-Entry

Table 3, the single-entry transaction matrix, and Table 4, the double-entry transaction matrix, contain all transactions for the first month of operations. At inception, the business

has neither capital nor liabilities. Indeed, no liability account is included in this example, although one could easily be added if required.

The first operating transaction is recorded in row `m1.w1`. During the week, the business earns 1,000 of revenue, incurs transportation expenses of 800, the owner withdraws 150 for personal use, and the remaining 50 is deposited into the bank account. The transactions for Weeks 2, 3, and 4 are identical to those of Week1.

Each row in both the single-entry and double-entry transaction matrices is equivalent to a journal entry. At first glance, what appears to be missing is the explanatory narrative that normally accompanies a journal entry. In the accompanying spreadsheet, this information is stored as an Excel note attached to each transaction row. Alternatively, the explanatory text could be placed directly in a dedicated column within the matrix. However, storing the information as a note is consistent with the broader concept that the transaction matrix can serve as a repository for both quantitative and qualitative transaction data. In this sense, each row contains not only the accounting flows associated with the transaction, but also the descriptive information necessary to explain its economic substance.

Following the operating transactions, the month-end adjusting and closing entries are recorded. The adjusting entry `m1.adj1` transfers the balance of the `own_draw` account to the owner's capital account. Since both accounts are balance-sheet accounts, this transfer is not required by accounting theory itself, but rather by the account specification adopted in this example.

At first glance, Table 4 appears considerably more complex than Table 3. However, the two representations contain the same underlying information, the principal difference is that the double-entry representation decomposes each variable into positive and negative components. In Table 3 positive values are input on the left side and negative on the right, so even for the double-entry transaction matrix one really does not have to consider debits and credits.

The primary difference between the two tables, aside from the fact that one is a single-entry representation and the other a double-entry representation, is that the double-entry transaction matrix explicitly displays debit and credit totals, whereas the single-entry transaction matrix does not. Apart from this decomposition, both tables represent the same underlying economic events.

Each matrix includes a *Select Report* checkbox associated with each time step. These checkboxes identify points in time at which financial reports may be generated. The role of these checkboxes will be discussed in greater detail in the following section on report generation.

Finally, the closing entry `m1.close` transfers the balances of the income-statement accounts (`revenue` and `transport`) to the owner's capital account. This resets the temporary income-statement accounts to zero and carries the month's net income forward into capital, thereby completing the accounting cycle for Month 1.

6.2 Stock Positions and Reports

SELECT REPORT	TIME
☐	inception
☐	m1.w1
☒	m1.w2
☐	m1.w3
☐	m1.w4

Figure 1: Selection of a reporting date from the transaction matrix.

In Table 3, the single-entry transaction matrix, and Table 4, the double-entry transaction matrix, there is a column labelled *Select Report*. Although the system permits multiple rows to be selected, only the first selected row is used for report generation.

When a row is selected, the system generates the account positions, income statement, and balance sheet corresponding to that point in time. Figure 1 illustrates the selection of the report associated with Month1, Week2 (m1.w2). The corresponding output is shown in Figure 2.

The report consists of three components. First, it displays the cumulative positions of all accounts at the selected time. Second, it presents the income statement for the period ending at that point. Third, it presents the corresponding intra-period balance sheet. Together, these reports provide a snapshot of both the flow activity and the stock positions of the accounting system at the selected time.

Positions, Income Statement and Balance Sheet

TIME : [M2.W2]

ACCOUNT	BANK	TRANSPORT	OWN_DRAW	REVENUE	CAPITAL
POSITIONS	340.00	1,760.00	-300.00	2,200.00	200.00

INCOME STATEMENT [m2.w2]		BALANCE SHEET [m2.w2 intra-period bal sheet]	
Revenue	2,200.00	Bank Bal	340.00
Transport	1,760.00	Liabilities	0.00
		Joes_Cap	200.00
		Own Draw	-300.00
		Curr. Period Profit	440.00
PROFIT	440.00	TOTAL	340.00 340.00

Figure 2: Positions, income statement, and balance sheet generated at m2.w2.

6.3 Intra-Period versus End-of-Period Balance Sheet

A useful distinction exists between an intra-period balance sheet and an end-of-period balance sheet. Conventionally, a balance sheet is presented in the form

$$\text{Assets} = \text{Liabilities} + \text{Equity}$$

where (A) denotes assets, (L) liabilities, and (E) equity. Since equity may be decomposed into capital and current-period profit,

$$\text{Equity} = \text{Capital} + \text{Profit},$$

it follows that

$$\text{Assets} = \text{Liabilities} + \text{Capital} + \text{Profit}.$$

For an intra-period balance sheet, current-period profit is generally non-zero and therefore appears explicitly as a separate component of equity. By contrast, an end-of-period balance sheet is prepared after the closing process, at which point current-period profit has been transferred to capital and the profit account has been reset to zero. Consequently, the

end-of-period balance sheet reduces to the more familiar form

$$\text{Assets} = \text{Liabilities} + \text{Capital}$$

The balance sheet shown in Figure 2 is therefore an intra-period balance sheet because current-period profit has not yet been closed to capital.

7 Traditional Journal and Ledger

The *Journal* and *Ledger* worksheets provide traditional accounting representations generated from the underlying transaction matrix.

These worksheets are not required for the operation of the framework, since all accounting information is contained within the transaction matrix itself. They are included primarily for comparison purposes and to demonstrate that conventional journal and ledger reports can be generated directly from the matrix representation. This illustrates the compatibility of the framework with traditional accounting practices while emphasizing that the transaction matrix serves as the primary accounting record.

8 Modern Spreadsheet Functions

The spreadsheet implementation uses the `LET` function, one of a family of modern spreadsheet functions introduced in recent versions of Excel. Although this workbook relies only on `LET`, related functions such as `LAMBDA`, `MAP`, `REDUCE`, and `SCAN` provide increasingly sophisticated programming capabilities within spreadsheets.

The use of `LET` is purely an implementation convenience. It allows complex formulas to be decomposed into named intermediate expressions, thereby improving readability, debugging, and maintenance. The accounting framework itself does not depend on these functions.

The use of `LET` requires a modern version of Microsoft Excel that supports dynamic-array functions.

Readers interested in this broader class of spreadsheet programming functions are referred to Brown (2026a) and Brown (2026b).

Repository Materials

The spreadsheet implementation, user guide, and related materials associated with this paper are available in the project repository:

<https://github.com/3sigmalectures-ctrl/recursive-partition-accounting>